

Real-time IoT Data Pipelines with yugabyteDB

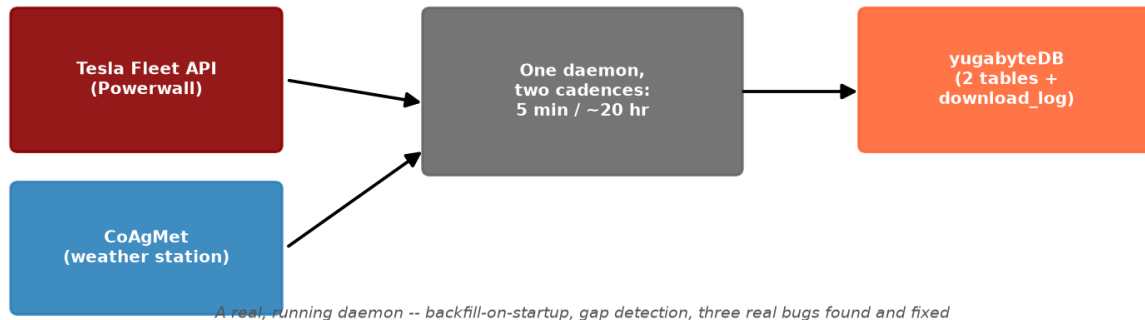


Figure 1: Figure 11-0: Real-time IoT Data Pipelines with yugabyteDB

Welcome to this edition of yugabyteDB Developer’s Notebook (YDN). This month we answer the following question(s);

I’ve been manually downloading data from a couple of IoT-style sources – a home battery system’s app, and a weather station’s web export – and hand-loading CSVs into yugabyteDB every so often. It’s tedious and error-prone. What does a real, unattended pipeline for this look like, end to end, including what breaks when you actually build one ?

This is a genuinely common shape of problem – one or more external APIs, each on their own update cadence, feeding a database that needs to end up complete and gap-free without a human in the loop. This month’s article is built entirely around a real daemon that replaced exactly this manual workflow: polling a home battery system’s Tesla Fleet API every five minutes, and a weather station’s daily export roughly once every twenty hours, into two yugabyteDB tables. Three real, non-obvious bugs surfaced while building it, each confirmed against the live external API rather than found by reading documentation – and each is the kind of thing that would silently corrupt data in a pipeline that wasn’t checked this carefully.

Software versions The primary software components used in this edition of YDN include yugabyteDB Anywhere (YBA) and yugabyteDB (YB) version 2026.1.1.0, Python 3, **psycopg2**, and **requests**. All of the steps below are run on one very large sized Mac Book Pro, or if you prefer, run these steps on yugabyteDB Aeon, yugabyteDB’s managed service, or Amazon Web Services (AWS), Google Cloud Platform (GCP), or another hyper-scaler.

For isolation and (simplicity), we develop and test all systems inside virtual machines or emulated machines, using the desktop hypervisors VMware Fusion version 13.6.4, and/or UTM version 4.6.5. Generally we run a single node of YBA, and 8 nodes of YB (5 nodes in one region, and 3 nodes in a second region) using Alma Linux versions 8 and 9, and Ubuntu Desktop version 24.04. We also run a client node using Ubuntu Desktop version 24.04.

11.1 Terms and core concepts

11.1.1 Introduction: Two Sources, Two Cadences, One Daemon

The pipeline in this article ingests from two genuinely different external sources, each on its own update rhythm, into two yugabyteDB tables:

- **Tesla Fleet API (a home Powerwall battery system)** – near-real-time. Polled every TESLA_POLL_SECONDS (300 seconds / 5 minutes), one row inserted per poll into `powerwall_metrics`.
- **CoAgMet (a Colorado State University agricultural weather station, station ftc01)** – updates once a day, a day behind the actual date. Checked every COAGMET_CHECK_SECONDS (20 hours), into `weather_data`.

Both run inside a single daemon process, on independent timers checked once a minute (`MAIN_LOOP_SECONDS` = 60) in one steady-state loop – not two separate cron jobs, so both sources share one database connection, one logging stream, and one failure-handling strategy.

Example 11-1: The schema being fed

```
CREATE TABLE powerwall_metrics
(
  recorded_at TIMESTAMP WITH TIME ZONE NOT NULL,
  home_kw DECIMAL(8,3),
  powerwall_kw DECIMAL(8,3),
  solar_kw DECIMAL(8,3),
  grid_kw DECIMAL(8,3),
  energy_remaining_pct DECIMAL(5,2),
  PRIMARY KEY (recorded_at DESC)
);

CREATE TABLE weather_data
(
  recorded_at DATE NOT NULL PRIMARY KEY,
  Avg_Temp_F DECIMAL, Max_Temp_F DECIMAL, Min_Temp_F DECIMAL,
  -- ... 11 more weather columns ...
);

CREATE TABLE download_log
(
  source TEXT PRIMARY KEY,
  last_success_at TIMESTAMPTZ,
  last_data_at TIMESTAMPTZ,
  notes TEXT
);
```

[!NOTE] PRIMARY KEY (`recorded_at DESC`) on `powerwall_metrics` is a deliberate choice, not a default – most-recent-first ordering matches how this data actually gets queried (dashboards, “what’s happening right now” checks), and it means a hash-sharded distributed primary key still serves the common access pattern efficiently. This is the same DESC-primary-key-for-recency pattern worth knowing from general yugabyteDB schema design, applied here to a real IoT feed rather than a synthetic example.

11.1.2 Startup Is Not “Just Start Polling” – It’s a Gap-Detection Query

A daemon that only polls going forward has a real problem: any period it wasn’t running (a crash, a deploy, a laptop closed overnight) becomes a permanent hole in the data. This pipeline handles that by treating every startup as a potential-gap event, not a fresh beginning. It runs `max(recorded_at)` directly against `powerwall_metrics` itself, finds the earliest incomplete or missing day since then, and backfills from there

Startup: gap detection is a query, not a log file

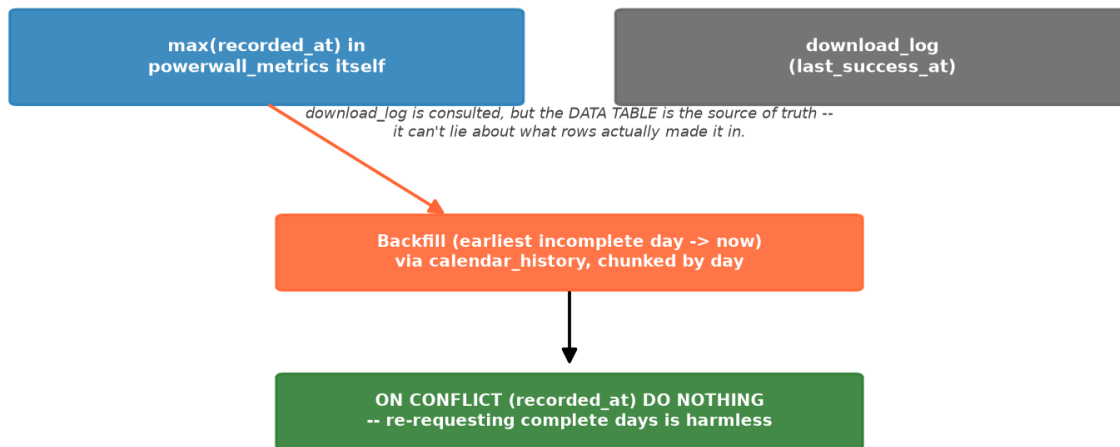


Figure 2: Figure 11-1: Startup gap detection is a query, not a log file

through now using the Tesla API's `calendar_history` endpoint (a completely different API call from the steady-state `live_status` poll) before ever entering the steady-state loop.

The `download_log` table exists purely as a bookkeeping optimization – but deliberately isn't the actual source of truth for what's missing. The comment in the daemon's own code states this design choice directly: *"it's consulted, but the real gap-detection source of truth is always `max(recorded_at)` in the data tables themselves (`download_log` could be behind if a batch partially failed; the data tables can't lie about what's actually there)."* This matters because a bookkeeping table can drift from reality (a crash mid-batch leaves it stale), while the data table's own contents cannot – querying what's actually there is always correct by construction.

Backfilled requests are also made idempotent deliberately, so a startup scan can never do harm even if it re-requests days that are already complete:

Example 11-2: Idempotent backfill insert

```
INSERT INTO powerwall_metrics
    (recorded_at, home_kw, powerwall_kw, solar_kw, grid_kw, energy_remaining_pct)
VALUES (%s, %s, %s, %s, %s, %s)
ON CONFLICT (recorded_at) DO NOTHING;
```

11.1.3 Three Real Bugs, Found by Running This Against a Live API

Each of the following was discovered by observing real, live behavior – not by reading the Tesla Fleet API's documentation, which does not mention any of them.

Bug 1 – comparing raw ISO timestamp strings across mixed UTC offsets. Tesla's `calendar_history` responses carry a fixed `-06:00` offset, while this pipeline's own internal `until` boundary is constructed in `+00:00`. String-comparing timestamps across differing offsets sorts incorrectly, because lexical string order and chronological order only agree when the offset is identical. Confirmed live: this silently dropped every row for 7 of 8 backfilled days before being caught – not an exception, not a

Three real bugs, found by running this against a live API

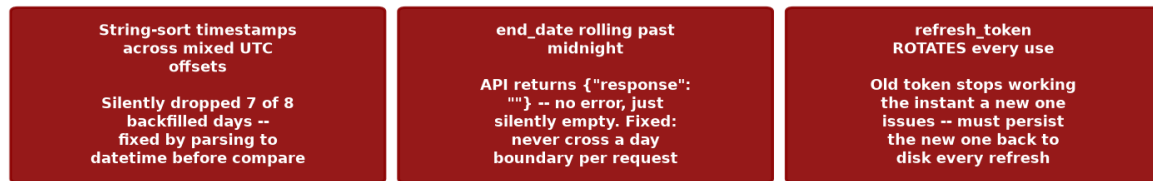


Figure 3: Figure 11-2: Three real bugs, found by running this against a live API

warning, just quietly incomplete data. The fix is to always parse to a real `datetime` object before comparing, which is correct regardless of offset:

Example 11-3: The fix

```
# WRONG -- string comparison, breaks across differing UTC offsets:
if ts <= since_str or ts > until_str:
    continue

# RIGHT -- parse first, compare as real datetimes:
ts_dt = datetime.fromisoformat(ts)
if ts_dt <= since or ts_dt > until:
    continue
```

Bug 2 – end_date rolling past midnight returns silent emptiness, not an error. The `calendar_history` endpoint is documented and tested against single-day granularity. The instant a request’s `end_date` rolls to the next day’s 00:00:00 – even by one second – the API returns `{"response": ""}` instead of an error or partial data. No exception is raised; there is nothing to catch. The fix, confirmed necessary through direct observation: every day’s backfill request must end at 23:59:59 of that same calendar day, never at the next day’s midnight, with backfill explicitly chunked one day at a time to enforce this.

Bug 3 – the OAuth refresh_token rotates on every single use. Confirmed empirically: the moment a new `refresh_token` is issued, the previous one stops working entirely. A daemon that reads a static token from a config file once at startup would work for one refresh cycle and then lock itself out permanently. The fix persists the new token back to `credentials.ini` immediately after every refresh, in place:

Example 11-4: Token rotation handling

```
def _refresh(self) -> None:
    resp = requests.post(self.creds["token_url"], data={
```

```

        "grant_type": "refresh_token",
        "client_id": self.creds["client_id"],
        "client_secret": self.creds["client_secret"],
        "refresh_token": self.creds["refresh_token"],
    }, timeout=30)
    resp.raise_for_status()
    body = resp.json()

    self.access_token = body["access_token"]
    self.access_token_expires_at = time.time() + body["expires_in"] - 300 # 5 min safety margin

    new_refresh_token = body["refresh_token"]
    if new_refresh_token != self.creds["refresh_token"]:
        persist_refresh_token(new_refresh_token) # write to disk immediately
        self.creds["refresh_token"] = new_refresh_token
        log("Tesla refresh_token rotated and persisted to credentials.ini")

```

[!NOTE] All three bugs share a pattern worth generalizing: none of them raised an exception. A string-sort bug silently drops rows; a day-boundary bug silently returns emptiness; a token-rotation miss would silently lock the daemon out on its next restart, with no crash to point at the cause. Building a pipeline against a live, imperfectly-documented third-party API means treating “it ran without an error” as necessary, not sufficient – each of these needed independent verification against the data actually received.

11.1.4 Deriving a Value That Isn’t Directly Provided

The Tesla API’s `live_status/calendar_history` responses report solar, grid, and battery power directly, but not home load – that has to be derived, and getting the sign conventions right matters:

Example 11-5: Deriving load, verified against a live sample

```

def derive_load_watts(point: dict) -> float:
    """load = solar + grid + battery + grid_services + generator, all raw
    signed as Tesla returns them. Verified against a live_status sample:
    solar=5240, battery=-2939 (charging), grid=0 => load=2301 (matched
    the live_status's own reported load_power exactly)."""
    return (
        point.get("solar_power", 0)
        + point.get("grid_power", 0)
        + point.get("battery_power", 0)
        + point.get("grid_services_power", 0)
        + point.get("generator_power", 0)
    )

```

This formula was not assumed correct because it looked reasonable – it was checked against a real `live_status` response that separately reports its own `load_power`, and the derived figure matched exactly. A negative `battery_power` means the battery is charging (drawing power in), which is why summing rather than subtracting the raw signed values produces the right answer.

The backfill path has a second wrinkle: `calendar_history` returns power readings on a 5-minute cadence but state-of-charge (`soe`) readings on a 15-minute cadence – different series, different timestamps. Since every power-series row needs a state-of-charge value, the pipeline forward-fills: for each power reading’s timestamp, it finds the most recent `soe` reading at or before that timestamp, rather than requiring an exact timestamp match that would only exist for one row in three.

11.1.5 Backups as a Byproduct, Not a Separate Job

On every startup, once caught up, the daemon writes all three tables (`powerwall_metrics`, `weather_data`, `download_log`) out to CSV – replacing what used to be a manual, monthly, hand-curated export process entirely. The disaster-recovery story is symmetric: those same generated CSVs are the documented restore path, not an afterthought bolted on separately. This is a small design choice worth calling out: rather than a separate backup job with its own schedule and its own chance to be forgotten, the backup is a natural byproduct of the same startup sequence that already has a fresh, live database connection open and has already confirmed the data is caught up.

11.2 Complete the following

In this section, we build and observe the two most instructive parts of this pipeline directly: the startup gap-detection/backfill sequence, and the steady-state dual-cadence poll loop.

11.2.1 Prerequisites

A yugabyteDB cluster reachable from a client machine, Python 3 with `psycopg2` and `requests` installed, and (to reproduce the Tesla-specific portions) your own Tesla Fleet API developer application credentials – registered directly with Tesla, not reused from anyone else’s account.

11.2.2 Step 1 – Create the schema

Example 11-6: Standing up the tables

```
psql -h <host> -U <user> -d <database> -f "20 - SQL/10 - Create table.sql"
```

This creates `powerwall_metrics`, `weather_data`, and `download_log` as shown in Example 11-1.

11.2.3 Step 2 – Configure credentials and run the daemon

Populate `properties.ini` (database connection) and `70 - Tesla Fleet API/credentials.ini` (OAuth client id/secret, refresh token, energy site id – obtained via Tesla’s own developer portal and OAuth flow, out of scope for this article to repeat). Then:

Example 11-7: Starting the daemon

```
python3 "72 - automated data download agent.py"
```

11.2.4 Step 3 – Observe the startup backfill

On first run against an empty `powerwall_metrics`, the daemon logs that it is skipping backfill (nothing to anchor to yet) and will start accumulating from the next live poll – confirm this matches section 11.1.2’s design: an empty table has no `max(recorded_at)` to measure a gap against. Stop the daemon, wait a few minutes, and restart it – this time, confirm it detects the resulting gap and backfills through `calendar_history` before resuming steady-state polling, exactly as described in section 11.1.2.

11.2.5 Step 4 – Observe the steady-state loop

Leave the daemon running and confirm, via its log output, that `tesla_poll_once` fires roughly every 5 minutes and a new row lands in `powerwall_metrics` each time, while the CoAgMet check log line only reappears on its own, much longer cadence – confirming the two independent timers in section 11.1.1 really do run independently within the single process.

Section summary In this section, we stood up the real schema behind this pipeline and ran the actual daemon end to end – watching it correctly skip backfill against an empty table, correctly detect and backfill a real gap on a subsequent restart, and correctly run two independently-timed sources inside one steady-state loop. Every behavior observed here traces back to a specific design decision explained in section

11.1 – gap detection against the data table itself, day-chunked idempotent backfill requests, and independent per-source timers sharing one process.

11.3 In this document, we reviewed or created

This month and in this document we detailed the following:

- A real, unattended daemon replacing a manual CSV-download workflow for two independent IoT-style data sources – a home Powerwall battery system polled every 5 minutes via the Tesla Fleet API, and a daily weather station export checked roughly every 20 hours – both landing in yugabyteDB on independent timers inside a single process.
- The startup gap-detection design: treating `max(recorded_at)` in the data table itself, not a separate bookkeeping log, as the true source of truth for what’s missing, followed by a day-chunked, idempotent (`ON CONFLICT DO NOTHING`) backfill.
- Three real, non-obvious bugs found only by running this against the live Tesla Fleet API – a UTC-offset string-comparison bug that silently dropped 7 of 8 backfilled days, a silent-empty-response gotcha when a date range crosses midnight, and an OAuth refresh token that rotates on every single use – each one that would have quietly corrupted or halted the pipeline without being independently verified against real data.
- A derived value (home load) checked against a real API response rather than assumed correct, and a forward-fill technique for merging two data series recorded on different cadences (5-minute power readings, 15-minute state-of-charge readings).

Persons who helped this month Kiyu Gabriel, David Bechberger

Additional resources: Free yugabyteDB training courses,

https://university.yugabyte.com/users/sign_in

Take any class, anytime, for free.

Jim Knicely’s very excellent blog site,

<https://yugabytedb.tips/>

This document is located here,

<https://www.yugabitten.com/62%20-%20Monthly%20articles/2026-11%20-%20Real-time%20IoT%20Data%20Pipeline%20Real-time%20IoT%20data%20pipelines.pdf>